The Real Deal

Wassim EL BAKHTAOUI
Yanis HAMMOUDA
Mathis CLAUDEL

2025/2026

INTRODUCTION **2**
PROBLEM SPACE -> SOLUTION SPACE **3**
Diagramme subdomain map 3
Hiérarchisation des Sous-Domaines (Core, Supporting, Generic) 4
Diagramme Context Map 5
Schémas Event Stroming 6
A. Recharge de portefeuille 6
B. Virer les gains du portefeuille vers le compte en banque du joueur : 6
C. récupérer les résultats d'un match : 7
D. Parier sur un match : 7
E. Renseigner une cote (Bookmaker) : 8
Diagramme UML d'architecture 9
RÉALISATION TECHNIQUE (Solution Space) **9**
Stack Technologique et Architecture 9
Description du fonctionnement de l'application 10
Etat du projet rendu 11
A. Couverture des exigences obligatoires 11
B. Bonus Techniques Implémentés 11
C. Bonus Cloud & Scalabilité (Étude d'architecture) 11
D. Elements manquants demandés via l'interview 12
E. Éléments Prototypés 13
Retour d'expérience 13
REPARTITION DES ROLES **13**
CONCLUSION **13**

INTRODUCTION

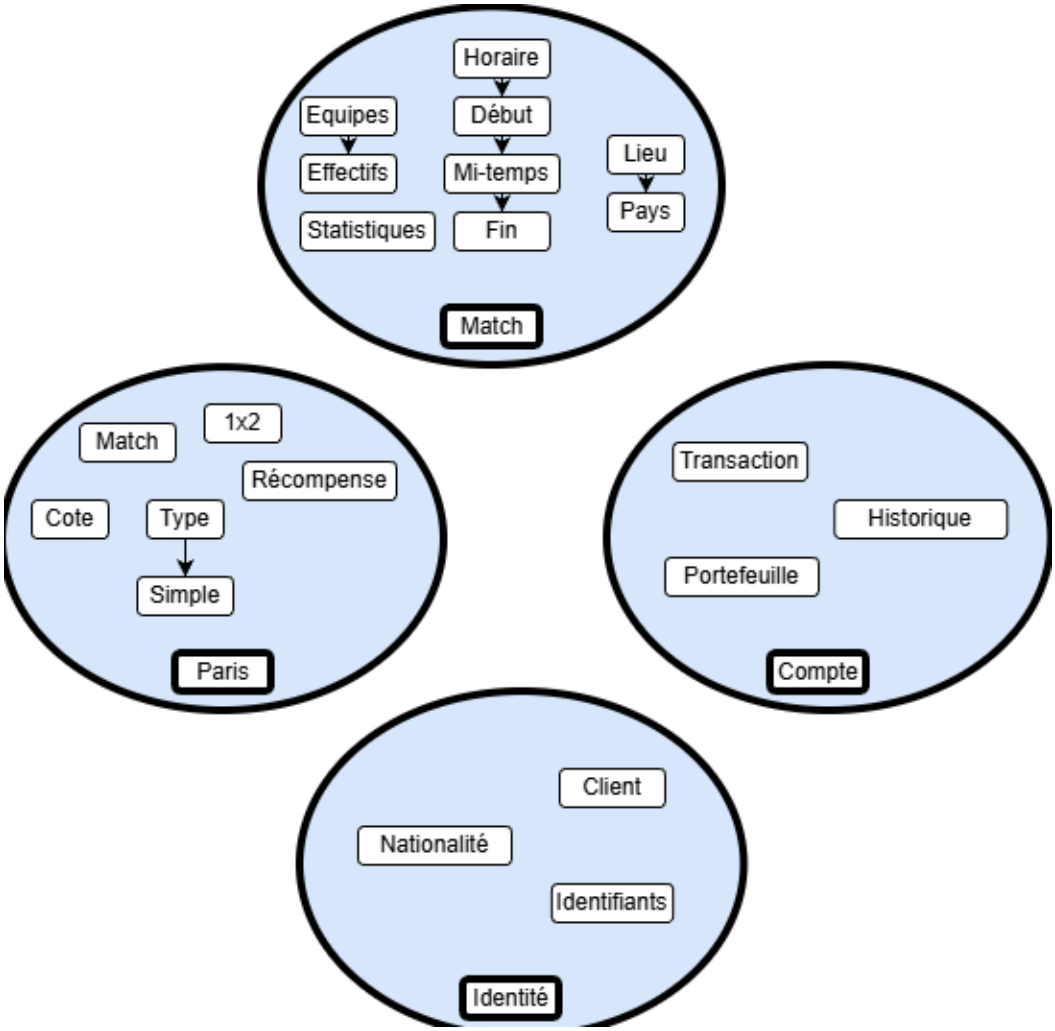
Pour ce projet de "Distribution du SI", nous avons dû réaliser une plateforme de paris sportifs. La difficulté principale du sujet n'était pas seulement de coder les fonctionnalités de base (parier, s'inscrire), mais surtout de concevoir une architecture capable de tenir la charge quand beaucoup d'utilisateurs se connectent en même temps.

Plutôt que de partir sur une application classique tout-en-un (monolithe), nous avons choisi de découper le système en plusieurs micro-services indépendants. L'idée est simple : séparer la gestion de l'argent de la gestion des matchs pour éviter que tout le système ne tombe si une partie est surchargée.

Ce rapport explique comment nous avons procédé. Nous présenterons d'abord notre analyse du sujet et le découpage des services (via la méthode DDD), puis l'architecture technique que nous avons mise en place avec Docker et RabbitMQ pour faire communiquer les différentes parties du projet.

PROBLEM SPACE -> SOLUTION SPACE

Diagramme subdomain map



Hiérarchisation des Sous-Domaines (Core, Supporting, Generic)

Core Subdomain :

Bet est classé dans Core car ce domaine constitue le cœur de l'application. C'est pour parier que les utilisateurs se rendent sur l'application, et c'est pour organiser les paris que les bookmakers travaillent.

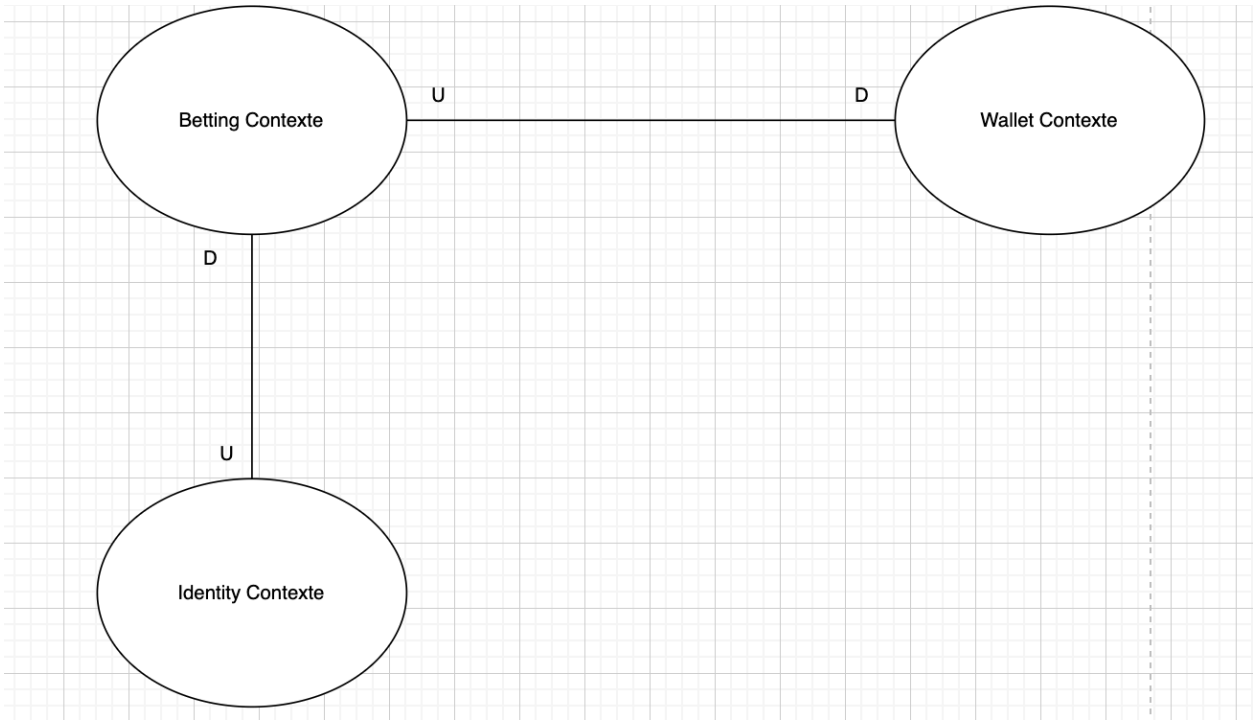
Supporting Subdomain :

Wallet, est dans notre cas, classé dans Supporting domain. Ce domaine participe à la réussite du domaine global en gérant les ressources des utilisateurs, ressources qui vont être jouées dans des paris.

Generic Subdomain :

Identity, est classé dans generic domain car il est nécessaire mais pas forcément différenciant. Il gère les comptes User et l'authentification.

Diagramme Context Map

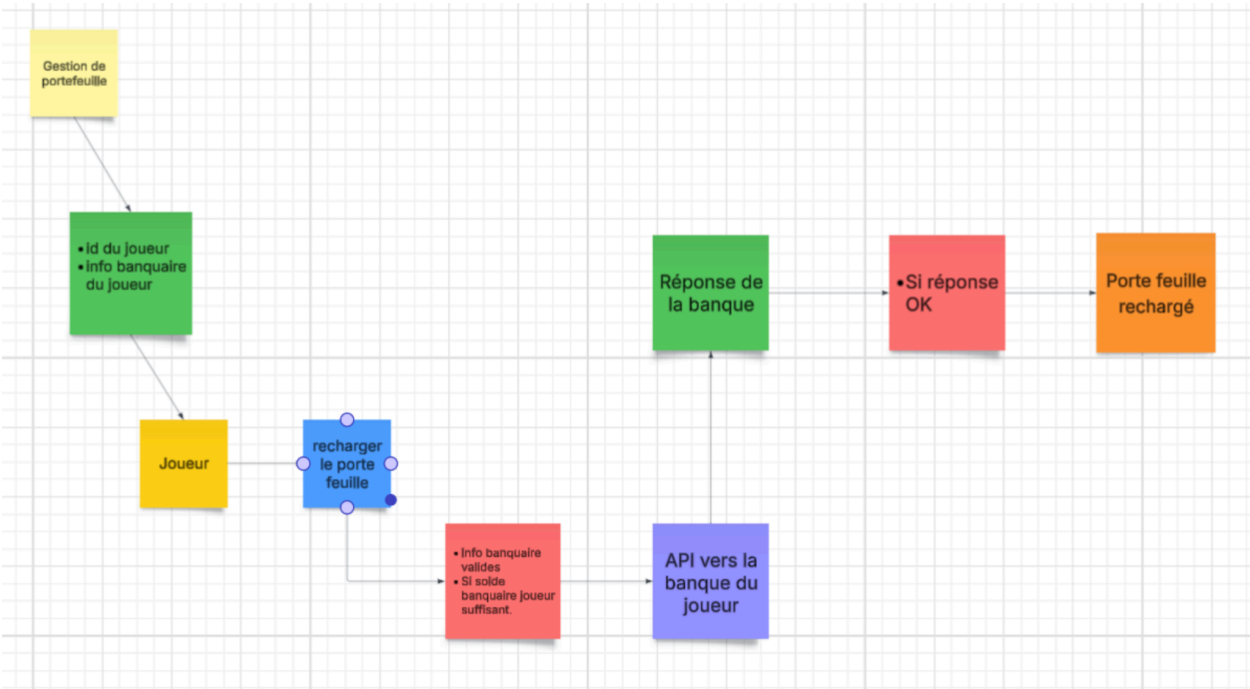


Identity est en position de Supplier (Fournisseur) : il fournit le service d'identification à tout le système. Les autres contextes sont donc ses clients (Customer).

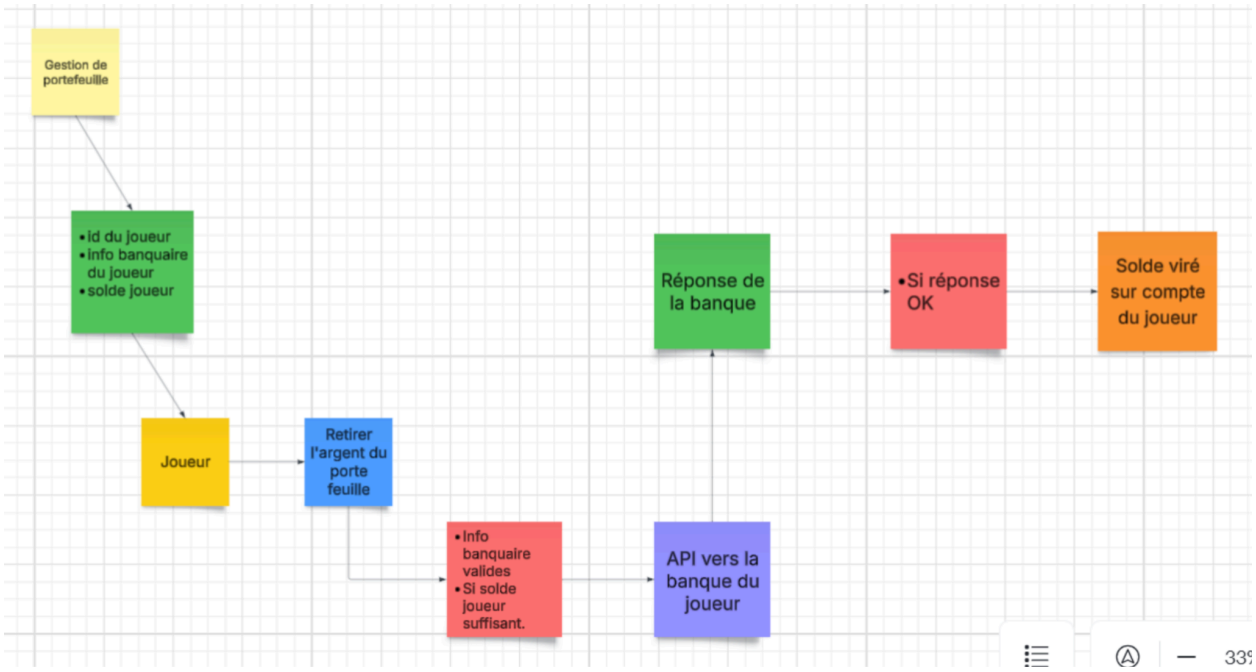
Entre le jeu et la finance, nous avons une relation Upstream/Downstream . Le contexte Betting est en Upstream car c'est lui qui décide si un pari est gagné ou perdu. Il envoie l'information au Wallet (Downstream), qui se contente d'exécuter l'ordre de débit ou de crédit. C'est ce choix qui justifie l'utilisation d'un bus d'événements (RabbitMQ) pour découpler les deux services.

Schémas Event Stroming

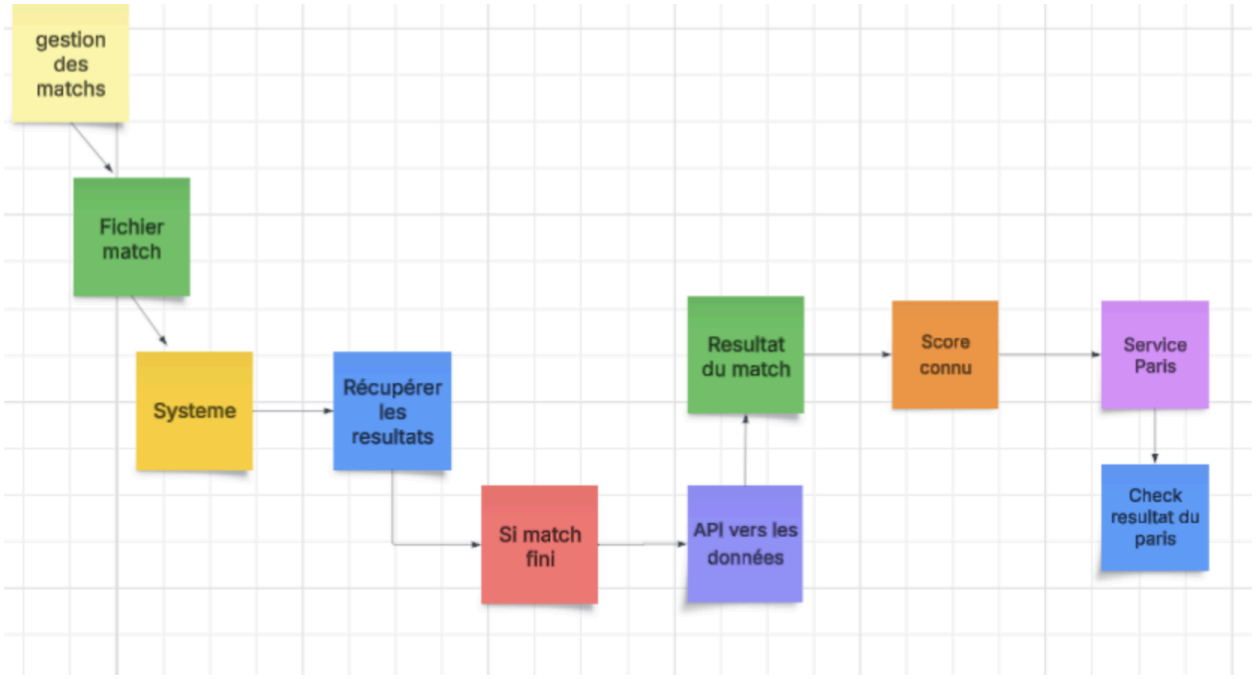
A. Recharge de portefeuille



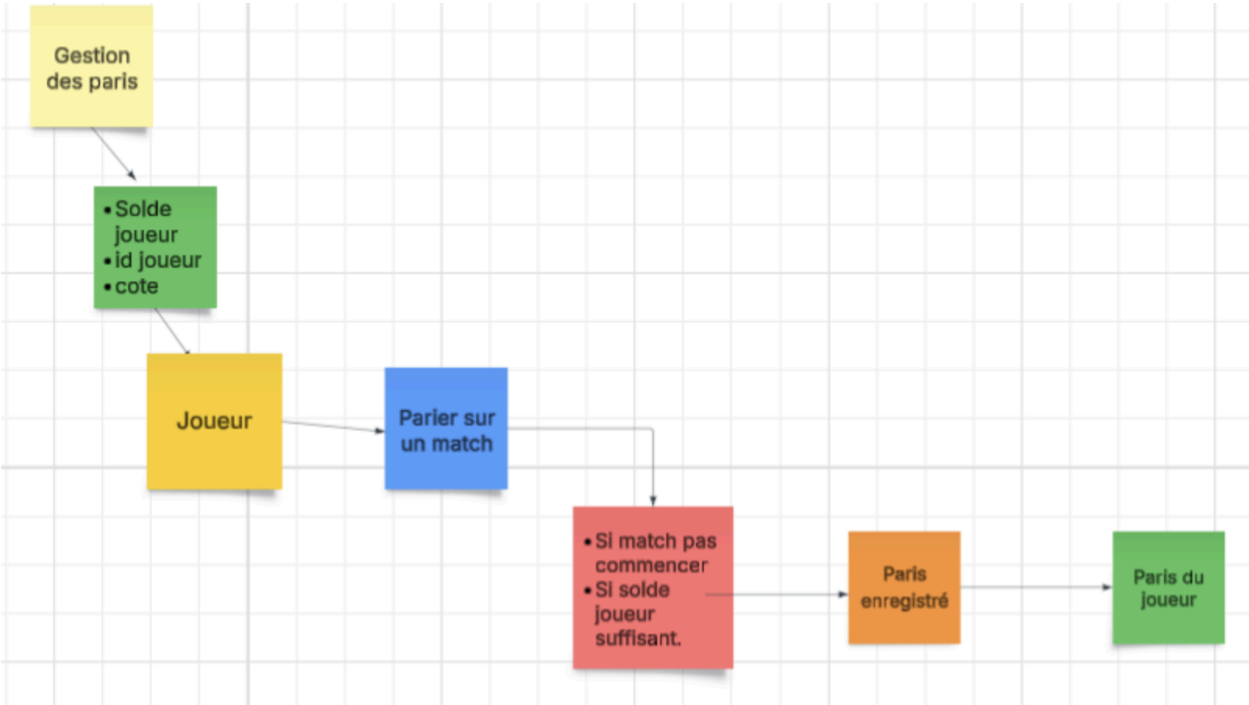
B. Virer les gains du portefeuille vers le compte en banque du joueur :



C. récupérer les résultats d'un match :

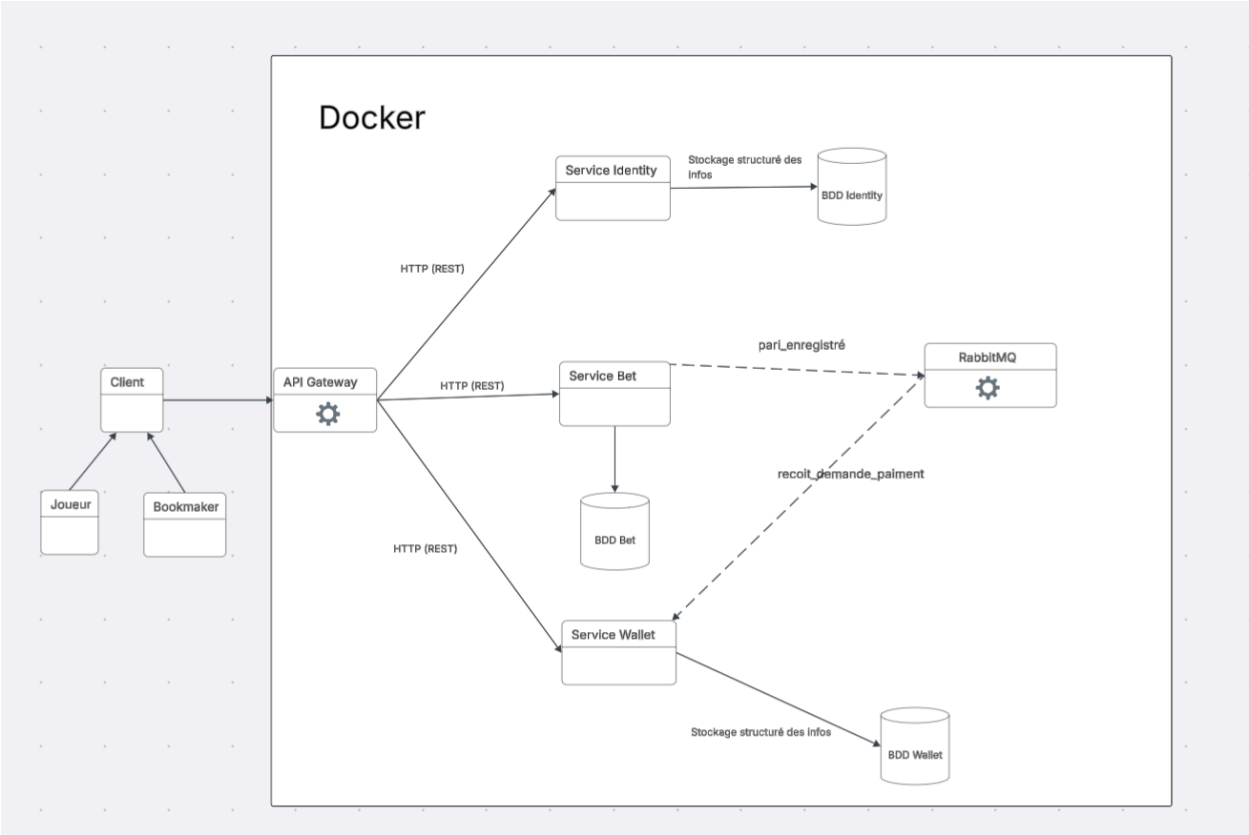


D. Parier sur un match :



E. Renseigner une cote (Bookmaker) :

Diagramme UML d’architecture



RÉALISATION TECHNIQUE (Solution Space)

Stack Technologique et Architecture

Pour répondre aux contraintes de charge et de distribution, on a opté pour une architecture **Microservices**.

- **Stack Technique :**
  - **Langage & Framework** : Au tout début du projet, on pensait partir sur du Spring Boot (Java), car c'est souvent la référence pour ce type d'architecture. Cependant, après en avoir discuté avec Monsieur Leroux, on a décidé de s'orienter plutôt vers NestJS (TypeScript). C'était le meilleur compromis : ça nous permettait de rester dans l'écosystème Node.js (plus léger et flexible) tout en gardant une structure rigoureuse (Injection de dépendances, Modules) très proche de ce que propose Spring ou Angular.
  - **Base de données :** Chaque microservice (Wallet, Betting, Identity) possède sa propre base PostgreSQL. Ça garantit une isolation totale des données (aucun service ne tape dans la base du voisin).
  - **Communication (Broker) :** Pour éviter que tout bloque en cas de pic de trafic, on a mis en place RabbitMQ. Ça nous permet de gérer les paris et les paiements de façon asynchrone, ce qui était un point crucial de la conception pour tenir la charge.
  - **Infra :** Tout le projet tourne sous Docker Compose. C'est super pratique pour lancer les 3 services, les 3 bases de données et le broker d'un seul coup sans avoir de conflits de versions sur nos PC.

Description du fonctionnement de l’application

Le système fonctionne avec trois briques principales qui communiquent entre elles :

1. **L'Authentification (Identity) :** L'utilisateur se connecte et reçoit un Token JWT. Ce "pass" est obligatoire pour discuter avec les autres services.
2. **Le Pari (Betting) :** Quand un joueur parie, le service vérifie juste si le match existe et si la cote est bonne. Si c'est OK, il ne bloque pas l'utilisateur : il envoie un message dans une file d'attente (RabbitMQ) pour dire qu’il faudrait débiter ce joueur.
3. **Le Portefeuille (Wallet) :** Ce service écoute la file d'attente. Quand il reçoit le message, il débit le compte. À la fin du match, c’est la même chose : le service Betting envoie un événement "Match Terminé" et le Wallet crédite les gagnants automatiquement.

Etat du projet rendu

A. Couverture des exigences obligatoires

Nous avons implémenté les éléments suivants conformément au sujet :

- **Services Métier & Données :** Développement de 3 services autonomes (*Identity, Wallet, Betting*) disposant chacun de sa propre base de données PostgreSQL isolée, garantissant l'indépendance des contextes.
- **Gateway & API :** Mise en place d'une API Gateway unique (NestJS) centralisant les entrées, redispatchant les requêtes vers les services internes et exposant une API REST documentée.
- **Cas d'utilisation :** Implémentation complète de deux flux majeurs : "Placer un pari" (Betting) et "Gérer son portefeuille" (Wallet), incluant la gestion de la cohérence des données entre ces deux entités.
- **Sécurité (Authn/Authz) :** Sécurisation globale via JWT. Le service *Identity* gère l'authentification (login/register) et les autres services valident les tokens via des Guards pour l'autorisation.
- **Tests :** Fourniture d'une collection de requêtes HTTP (scripts Curl/Postman) permettant de valider les scénarios nominaux et les cas d'erreur.

B. Bonus Techniques Implémentés

Nous avons choisi d'implémenter les bonus liés à l'infrastructure locale et à la résilience :

- **Conteneurisation (Docker) :** Définition de Dockerfile optimisés (multi-stage builds) pour chaque service. Orchestration complète via un fichier docker-compose.yml unique gérant les réseaux, les volumes persistants et les dépendances au démarrage.
- **Message Broker (RabbitMQ) :** Intégration d'un bus d'événements AMQP pour la communication asynchrone. Cela permet de découpler les services *Betting* et *Wallet*, assurant que la prise de pari reste fluide même en cas de latence sur les transactions financières.

C. Bonus Cloud & Scalabilité (Étude d'architecture)

Concernant le bonus sur le Cloud Provider, nous avons réalisé l'étude d'architecture cible pour un déploiement "Production" capable de supporter la charge d'une Coupe du Monde, bien que le déploiement effectif n'ait pas été finalisé pour ce rendu.

- **Sélection du Provider :** Nous avons retenu **AWS (Amazon Web Services)** pour sa maturité sur les services managés.
- **Identification des services (Mapping IaaS/PaaS) :**
    - *Compute :* Migration des conteneurs Docker vers **AWS ECS (Elastic Container Service)** avec **Fargate**(Serverless Compute) pour l'auto-scaling automatique des pods en fonction de la charge CPU/RAM.
    - *Database :* Remplacement des conteneurs Postgres locaux par **Amazon RDS for PostgreSQL** (PaaS) pour gérer les sauvegardes et la réplication Multi-AZ (Haute Disponibilité).
    - *Broker :* Remplacement du conteneur RabbitMQ par **Amazon MQ** (service managé pour RabbitMQ).
- **Estimation des coûts :** Pour une durée de 3 mois (période Coupe du Monde), avec un trafic estimé élevé nécessitant plusieurs instances de l'API Gateway et du service Betting, le coût principal résiderait dans les instances RDS et le trafic sortant (Data Transfer Out), justifiant l'usage potentiel de Spot Instances pour réduire la facture sur les nœuds de calcul non critiques. On a pas le prix exact car on a pas eu le temps ni même les crédits (car on a utilisé les credits student pour un autre cours) pour avoir une estimation exacte des coûts.

D. Elements manquants demandés via l’interview

Suite à l'entretien avec l'expert métier, nous avons dû opérer des choix stratégiques pour respecter les délais. Par exemple, bien que la gestion des paris combinés ait été évoquée, nous avons décidé de nous concentrer exclusivement sur le pari simple (1N2) pour ce MVP. La complexité de gérer des combinés dans une architecture distribuée asynchrone était trop risquée techniquement pour une première version ; nous avons donc privilégié la robustesse du flux de paiement (RabbitMQ/Wallet) sur la richesse fonctionnelle. De même, concernant la récupération des scores, nous n'avons pas connecté d'API sportive externe (type *MatchEnDirect*) pour éviter de dépendre d'un service tiers payant ou instable lors de la démonstration. Nous déclenchons la fin des matchs manuellement via une route API dédiée, ce qui garantit la testabilité du système.

E. Éléments Prototypés

L'interaction avec un système bancaire réel (type Stripe) est prototypée. Une route interne simule la validation bancaire pour permettre le test fonctionnel du dépôt d'argent.

Retour d’expérience

Le plus dur a été de changer notre façon de penser pour l'asynchrone. Faire communiquer le *Betting* et le *Wallet* sans qu'ils soient connectés à la même base de données a posé des soucis de cohérence au début (ex: comment être sûr que le débit est passé ?). La solution trouvée est qu’on a mis en place des DTOs (objets de transfert de données) stricts pour que les messages RabbitMQ soient toujours bien formatés. L’utilisation du docker nous aidera pour éviter tout soucis lors de l’évaluation du projet afin qu’il puisse être lancé sans problème sur un autre ordinateur.

REPARTITION DES ROLES

Wassim EL BAKHTAOUI	Toute la partie code ainsi que la partie rapport sur la programmation
Yanis HAMMOUDA	Les 5 events stormings, Product Owner avec la synthèse de l'interview, Diagramme d'architecture, aide mise en place du docker et classification des domaines.
Mathis CLAUDEL	Subdomain map

CONCLUSION

En conclusion, ce projet *The Real Deal* nous a vraiment permis de voir le fossé qu'il y a parfois entre la théorie et la pratique, surtout quand on s'attaque à des architectures distribuées. Le passage aux Microservices a été un défi technique pour nous, notamment pour réussir à faire communiquer le service de paris et le wallet via RabbitMQ sans perdre de données en route, mais c'est clairement là-dessus qu'on a le plus progressé. Au final, on est plutôt fiers du résultat parce qu'on a un backend qui tient la route et une infra Docker qui tourne proprement. Si on avait eu un peu plus de temps, la suite logique aurait été de déployer tout ça sur un vrai cluster Cloud comme AWS et de développer une interface graphique pour rendre le projet plus visuel, mais ça reste une super expérience pour maîtriser les standards actuels comme NestJS et l'asynchrone.